

COMMON COURSE ASSIGNMENT

A. Assignment Information

Particular	Details
Department:	CSE(AI)
Programme:	COMPUTER SCIENCE AND ENGINEERING
Course Code & Course Name	DSA0405 – FUNDAMENTALS OF DATA SCIENCE
Academic Year / Batch	2024
Faculty Name	DR KRISHNA RAJ MONY
Assignment Title	COMMON COURSE ASSIGNMENT
Date of Issue	30-08-26
Date of Submission	03-09-26
Maximum Marks	100
Course Outcome(s) – CO	CO5
Bloom's Taxonomy Level	BL5
SDG Mapping (SDG 1-17)	SDG-4
Industry / Societal Relevance	Early identification of academically at-risk students can help institutions provide timely support, improve learning outcomes, and support data-driven academic interventions.

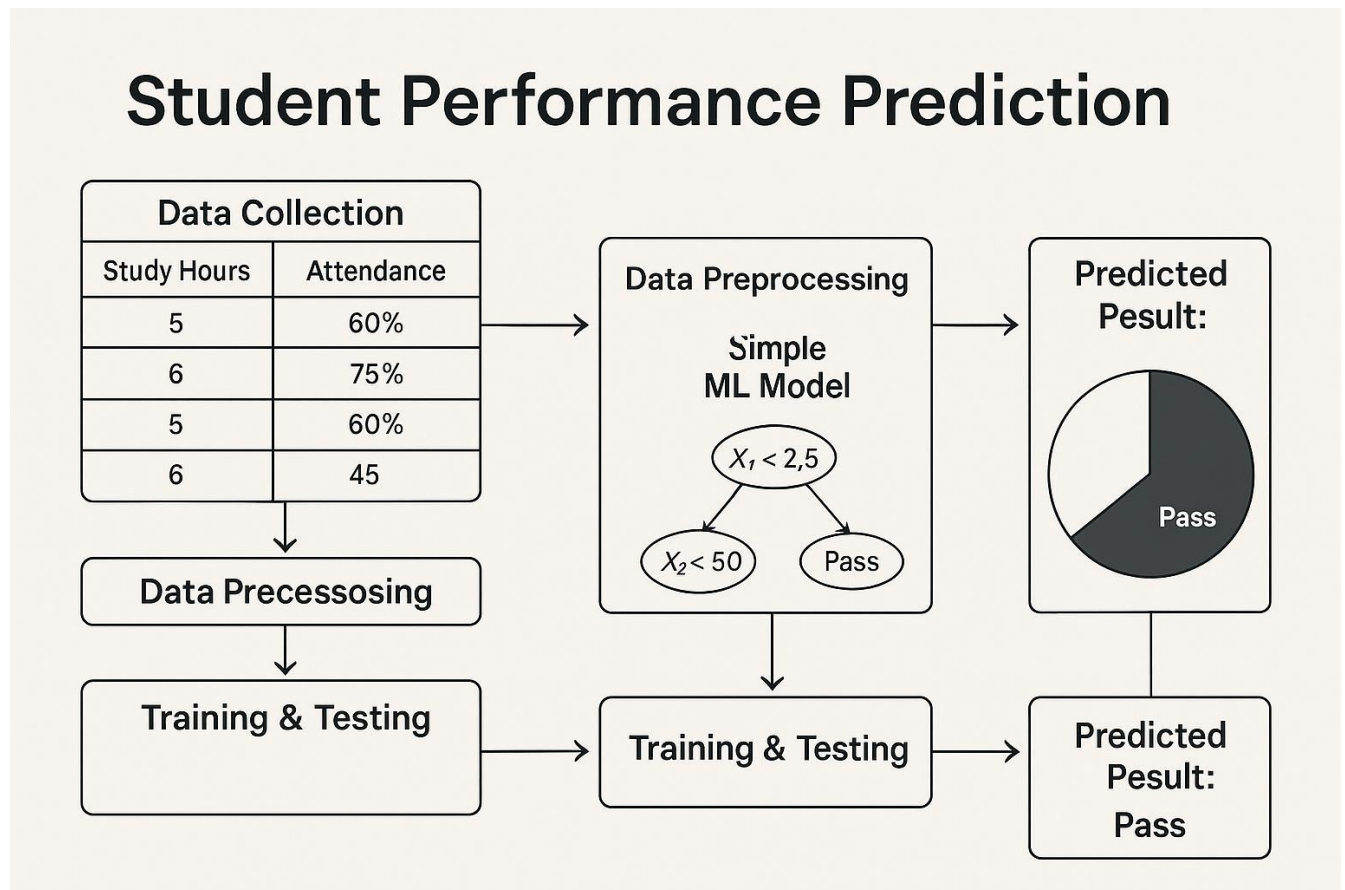
B. Assignment Problem / Challenge

Student Performance Risk Prediction :

Develop a machine-learning-based student performance risk predictor using attendance, past grades, and study hours. Apply k-Nearest Neighbors (kNN) classification to classify students into “At Risk” and “Safe” categories and interpret the prediction results for early academic intervention.

1. Problem Statement and Problem Formulation**Problem Statement**

Educational institutions often need to identify students who may be at risk of poor academic performance. Attendance, previous grades, and study habits can provide useful information about a student's academic situation. However, examining these factors manually for a large number of students can be time-consuming and may not provide consistent risk identification.



The proposed project aims to classify students into meaningful academic risk categories using the k-Nearest Neighbors (kNN) classification algorithm. The model uses attendance percentage, past grades, and study hours to determine whether a student is likely to be “At Risk” or “Safe.” The prediction can help teachers and academic advisors identify students who may require additional support.

Problem Formulation

Let each student be represented by a feature vector:

$$\mathbf{X}_i = [A_i, G_i, S_i]$$

where:

- A_i = Attendance Percentage of student i
- G_i = Past Grade / Previous Academic Score of student i
- S_i = Study Hours of student i

The objective is to classify each student into one of two classes: At Risk or Safe, based on the similarity of the student's feature vector to labeled training students.

For a new student x , kNN identifies the k nearest training samples using a distance measure such as Euclidean distance. The majority class among these neighbors is assigned as the predicted risk category.

2. Objective and Expected Outcomes

Objectives

1. To collect academic and study-related data for students.
2. To select important indicators such as attendance, past grades, and study hours.
3. To preprocess and standardize the student data.
4. To divide the dataset into training and testing sets.
5. To apply the k-Nearest Neighbors (kNN) classification algorithm.
6. To test different values of k and select an appropriate value based on validation performance.
7. To classify students into At Risk and Safe categories.
8. To evaluate the classification model using suitable performance metrics.
9. To visualize and interpret the prediction results.
10. To identify limitations and possible improvements to the predictor.

Expected Outcomes

- Students will be classified into At Risk or Safe categories.
- The model will identify patterns between attendance, past grades, study hours, and academic risk.
- Different k values will be compared to determine a suitable neighborhood size.
- Performance metrics such as accuracy, precision, recall, and F1-score will be obtained.
- A confusion matrix will show correct and incorrect classifications.
- The analysis can support early academic intervention and student support.

3. Requirements, Constraints and Assumptions

Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Internet connection for dataset collection or library installation

Software Requirements

- Python 3.x
- Jupyter Notebook / Google Colab / VS Code
- Pandas
- NumPy
- Matplotlib
- Scikit-learn
- Seaborn

Dataset Requirements

The dataset should contain student records with attributes such as:

Student	Attendance (%)	Past Grade	Study Hours
S001	92	86	3.5
S002	68	55	1.5
S003	85	78	3.0
S004	60	48	1.0
S005	95	91	4.0

3. Requirements, Constraints and Assumptions (Continued)

Constraints

- The quality of prediction depends on the quality and completeness of student records.
- The model should use relevant academic and study-related features.
- Training and testing data must be properly separated.
- Different values of k should be tested rather than assuming a single value is always optimal.
- Feature scaling should be considered because kNN is distance-based.
- Class imbalance, if present, should be considered during evaluation.
- Student academic information must be handled with appropriate privacy and ethical safeguards.

Assumptions

- Attendance, past grades, and study hours provide useful information for academic risk prediction.
- Missing values are handled before model training.
- Numerical features are standardized before applying kNN.
- The training data is representative of the students to whom the model will be applied.
- The risk labels used for supervised learning are available from institutional records or an appropriate labeling rule.
- The prediction is used as a support tool and not as a final judgment about a student.

4. Application of Relevant Course Knowledge / Concepts

The project applies several concepts from Data Science and Machine Learning.

1. Data Preprocessing

Missing values, duplicate records, inconsistent values, and unsuitable entries are identified and handled before model training.

2. Feature Selection

- Attendance Percentage
- Past Grade / Previous Academic Score
- Study Hours

3. Feature Scaling

Since the selected features may have different ranges, StandardScaler is used so that no feature dominates the distance calculation.

$$z = (x - \mu) / \sigma$$

4. Supervised Learning

kNN is a supervised classification algorithm because the training dataset contains predefined class labels such as At Risk and Safe.

5. Distance-Based Classification

kNN uses distance between observations to determine similarity. Euclidean distance can be expressed as:

$$d(x,y) = \sqrt{\sum (x_j - y_j)^2}$$

4. Application of Relevant Course Knowledge / Concepts (Continued)

6. Hyperparameter Selection

The value of k determines how many neighboring observations participate in the classification. Small k values can be sensitive to noise, while large k values can smooth local patterns. Values such as k = 3, 5, 7, and 9 can be compared.

7. Model Evaluation

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

8. Data Visualization

Bar charts, scatter plots, confusion-matrix heatmaps, and performance comparison graphs can be used to understand the dataset and model results.

5. Design / Proposed Solution / Methodology

The proposed system follows this pipeline:

Dataset Collection



Data Cleaning



Feature Selection



Feature Scaling



Train/Test Split



Test Different k Values



Apply kNN



Predict Risk Category



Evaluate Model



Analyze and Interpret Results

Proposed Risk Classification

Category	Typical Characteristics	Interpretation
----------	-------------------------	----------------

At Risk	Low attendance and/or low past grades and/or limited study hours	Student may require additional academic support
Safe	Adequate attendance, stronger past grades, and/or sufficient study hours	Student is less likely to be classified as academically at risk

6. Algorithm / Pseudocode / Flowchart

Algorithm – Student Risk Classification Using kNN

1. Load the student performance dataset.
2. Select attendance, past grade, and study hours.
3. Check and handle missing or invalid values.
4. Encode the target label as At Risk or Safe.
5. Standardize the selected numerical features.
6. Split the data into training and testing sets.
7. Evaluate several k values such as 3, 5, 7, and 9.
8. Train the kNN classifier using the selected k.
9. Predict the risk category for test students.
10. Calculate accuracy, precision, recall, and F1-score.
11. Generate a confusion matrix and relevant graphs.
12. Analyze the results and identify possible improvements.

Pseudocode

START

Load student performance dataset

Select:

Attendance

Past Grade

Study Hours

Risk Label

Handle missing values

Split data into training and testing sets

Standardize numerical features

FOR k = 3, 5, 7, 9:

Train kNN classifier

Predict test labels

Calculate accuracy
Select the best-performing k
Train final kNN model using selected k
Predict At Risk / Safe
Calculate precision, recall and F1-score
Generate confusion matrix
Generate graphs
Display results
END

Flowchart

START



Load Dataset



Clean Data



Select Features



Scale Features



Train/Test Split



Select Optimal k



Apply kNN



Predict Risk



Evaluate & Visualize



END

7. Implementation / Source Code and Environment / Tools

Used Environment

- Programming Language: Python
- Development Environment: VS Code / Jupyter Notebook / Google Colab
- Libraries: Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn

Source Code

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, ConfusionMatrixDisplay
)

data = pd.read_csv("student_performance.csv")

features = ["Attendance", "Past_Grade", "Study_Hours"]
target = "Risk"

X = data[features].copy()
y = data[target].copy()

X = X.fillna(X.median())

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

scaler = StandardScaler()
```

```

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

results = []

for k in [3, 5, 7, 9]:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train_scaled, y_train)
    pred = model.predict(X_test_scaled)

    results.append({
        "k": k,
        "Accuracy": accuracy_score(y_test, pred),
        "Precision": precision_score(y_test, pred, pos_label="At Risk"),
        "Recall": recall_score(y_test, pred, pos_label="At Risk"),
        "F1": f1_score(y_test, pred, pos_label="At Risk")
    })

results_df = pd.DataFrame(results)
print(results_df)

best_k = results_df.loc[
    results_df["F1"].idxmax(), "k"
]

final_model = KNeighborsClassifier(n_neighbors=int(best_k))
final_model.fit(X_train_scaled, y_train)

final_pred = final_model.predict(X_test_scaled)

print("Best k:", best_k)
print("Accuracy:", accuracy_score(y_test, final_pred))
print("Precision:", precision_score(

```

```

    y_test, final_pred, pos_label="At Risk"
))
print("Recall:", recall_score(
    y_test, final_pred, pos_label="At Risk"
))
print("F1-score:", f1_score(
    y_test, final_pred, pos_label="At Risk"
))

cm = confusion_matrix(
    y_test, final_pred, labels=["At Risk", "Safe"]
)
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["At Risk", "Safe"]
)
disp.plot()
plt.title("kNN Confusion Matrix")
plt.show()

```

Tools and Purpose

Tool	Purpose
Python	Model implementation
Pandas	Dataset loading and processing
NumPy	Numerical operations
Scikit-learn	Scaling, kNN classification and evaluation
Matplotlib	Graphs and confusion matrix visualization
Seaborn	Statistical visualization

8. Test Cases and Expected/Actual Results

Test Case	Input/Condition	Expected Result	Actual Result
TC01	Load valid student dataset	Dataset loads successfully	To be recorded after execution
TC02	Check missing values	Missing values identified	To be recorded after execution
TC03	Standardize features	Features converted to comparable scale	To be recorded after execution
TC04	$k = 3$	kNN model generates predictions	To be recorded after execution
TC05	$k = 5$	kNN model generates predictions	To be recorded after execution
TC06	$k = 7$	kNN model generates predictions	To be recorded after execution
TC07	$k = 9$	kNN model generates predictions	To be recorded after execution
TC08	Predict student risk	Student classified as At Risk or Safe	To be recorded after execution
TC09	Generate confusion matrix	Confusion matrix displayed	To be recorded after execution
TC10	Calculate performance metrics	Accuracy, precision, recall and F1-score displayed	To be recorded after execution

9. Execution Screenshots / Outputs

The following screenshots should be included in the final report after running the program:

Screenshot 1 – Dataset

Dataset Summary & Raw Records

Filter by Risk Status:

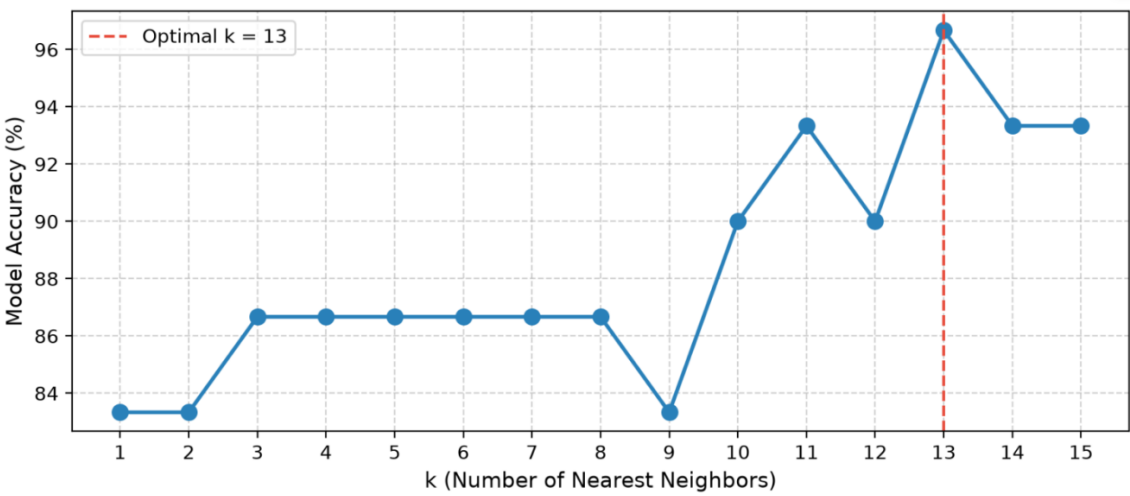
At Risk x Safe x

	Attendance	Past_Grade	Study_Hours	Risk
0		64.9	90	1 Safe
1		95.4	53.2	5.3 Safe
2		83.8	48	5.4 Safe
3		76.7	66.9	6.2 Safe
4		53.3	94.2	7 Safe
5		53.3	53.3	9.3 At Risk
6		48.1	77	5.1 At Risk
7		90.9	81.9	3.4 Safe
8		76.9	53.1	7.7 Safe
9		82.5	80.1	2.9 Safe

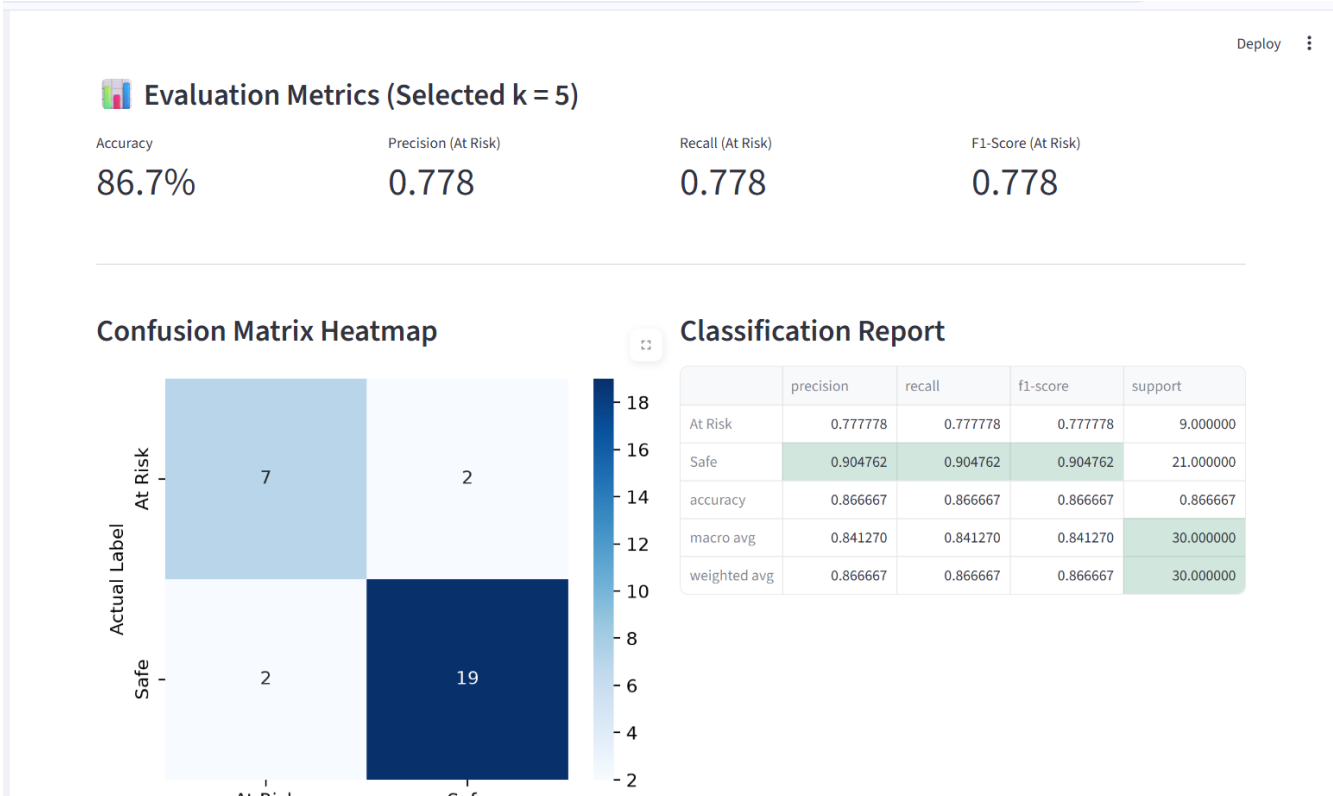
Screenshot 2 – k Value / Performance Comparison

k Value vs. Test Accuracy

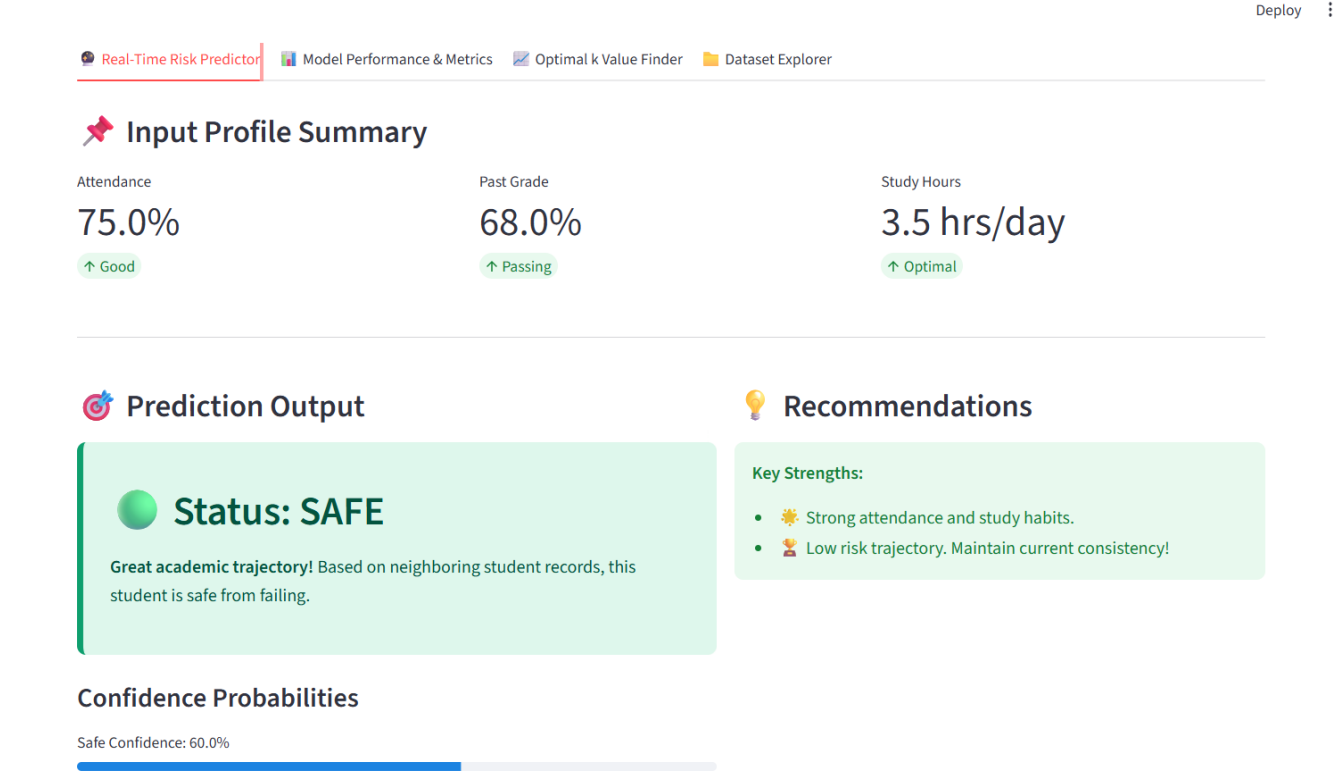
Tested $k \in [1, 15]$. Best Accuracy (96.7%) is achieved at $k = 13$.



Screenshot-3 Confusion Matrix



Screenshot 4 – Final Student Risk Classification



10. Results and Validation

The kNN model should be validated by testing its classification performance on unseen test data. The final result should identify whether students are classified as At Risk or Safe based on the selected academic features.

Result Table

Risk Category	Typical Attendance	Typical Past Grade	Typical Study Hours	Interpretation
At Risk	Lower	Lower	Lower/limited	May require academic intervention
Safe	Adequate/higher	Adequate/higher	Adequate/higher	Less likely to be at risk
Final Model	To be obtained	To be obtained	To be obtained	Based on model predictions

Performance Metrics

1. Accuracy – proportion of correctly classified students.
2. Precision – proportion of students predicted as At Risk who are actually At Risk.
3. Recall – proportion of actual At Risk students correctly identified.
4. F1-score – harmonic mean of precision and recall.
5. Confusion Matrix – shows correct and incorrect predictions for both classes.

Validation

- Compare performance for multiple k values.
- Check the confusion matrix for false positives and false negatives.
- Verify that the model is evaluated on unseen test data.
- Check whether the model performs consistently when the train/test split is changed.
- Review whether the predicted risk categories are meaningful for academic support.

11. Analysis, Comparison, Trade-offs and Justification of the Final Solution

Comparison of Different k Values

k	Advantages	Disadvantages
3	Captures local patterns	More sensitive to noise
5	Balanced neighborhood size	May still be affected by local variation
7	Smoother classification	Can reduce sensitivity to smaller patterns
9	More stable in noisy data	May oversmooth local differences

11. Analysis, Comparison, Trade-offs and Justification of the Final Solution (Continued)

Trade-offs

kNN Advantages:

- Simple and easy to understand.
- Suitable for binary classification problems such as At Risk/Safe.
- No complex training process is required.
- Can work well when similar students have similar risk categories.
- The result can be interpreted through neighboring training samples.

kNN Limitations:

- Prediction can be slower for large datasets because distances to training samples are calculated.
- Sensitive to feature scaling.
- Sensitive to irrelevant features and noisy observations.
- Choice of k affects prediction performance.
- Class imbalance may affect majority voting.
- The model's performance depends strongly on the quality of the training data.

Justification

kNN is selected because the project requires classification of students into predefined risk categories and the available input variables are numerical and suitable for distance-based comparison. Standardization ensures that attendance, past grades, and study hours contribute appropriately to the distance calculation. The final k should be selected using validation results rather than assuming that one value is universally optimal.

The predictor should be treated as an academic support system. A classification result should not be used as the sole basis for decisions affecting a student's academic progression.

12. Broader Considerations / SDG Relevance

SDG 4 – Quality Education

The project directly supports Quality Education by helping educational institutions identify students who may require additional academic assistance. Early identification can support tutoring, mentoring, and targeted learning interventions.

Ethical Considerations

- Student academic information should remain confidential.
- Predictions should not be used to unfairly label or penalize students.
- Students should have access to appropriate academic support after risk identification.
- Model limitations and uncertainty should be communicated to decision-makers.

Social Considerations

A risk prediction system can help teachers and academic advisors allocate support resources more effectively, particularly when the number of students is large.

12. Broader Considerations / SDG Relevance (Continued)

Privacy

Personal identifiers should be removed or protected when student records are used for analysis. Access to academic data should be limited to authorized users.

Accessibility

The system should provide clear and understandable risk results so that teachers and academic advisors can use the information without requiring advanced machine-learning knowledge.

Professional Responsibility

Developers should ensure that the model is tested properly and that its predictions are not presented as certain outcomes. Academic decisions should consider additional evidence and human judgment.

13. Conclusion, Limitations and Possible Improvements

Conclusion

The project demonstrates how the k-Nearest Neighbors (kNN) classification algorithm can be used to predict student academic risk using attendance, past grades, and study hours. After preprocessing and feature scaling, the model classifies students into At Risk and Safe categories.

The comparison of different k values and evaluation using accuracy, precision, recall, F1-score, and a confusion matrix provides a structured way to validate the predictor. The system can support early identification of students who may benefit from academic intervention while keeping final decisions under appropriate human supervision.

Limitations

1. The prediction quality depends heavily on the quality and size of the dataset.
2. Only a limited number of academic and behavioral indicators may be included.
3. kNN is sensitive to feature scaling and the selected k value.
4. Irrelevant features and outliers can affect distance calculations.
5. Risk labels may depend on the institutional definition of academic risk.
6. The model cannot fully represent personal, social, financial, or learning factors that may influence performance.

7. A prediction should not be interpreted as a definitive statement about a student's future performance.

Possible Improvements

- Include additional indicators such as assignment completion, internal assessment marks, previous semester performance, and learning activity.
- Use a larger and more diverse institutional dataset.
- Compare kNN with other suitable classification algorithms.
- Use cross-validation for more reliable model selection.
- Investigate class-balancing methods when the classes are imbalanced.
- Develop a dashboard for teachers and academic advisors.
- Periodically retrain and validate the model as student patterns change.

14. Student Reflection

Students should reflect on the implementation and answer the following questions:

1. What did you learn from implementing the kNN classification algorithm?
2. How did feature scaling affect the kNN model?
3. How did changing the value of k affect prediction performance?
4. What challenges did you face while preprocessing the student dataset?
5. How did the confusion matrix help you understand the model's errors?
6. If additional time and resources were available, what improvements would you make and why?

15. References

1. UCI Machine Learning Repository – Student Performance dataset.
2. Scikit-learn documentation – k-Nearest Neighbors classification.
3. Python documentation.
4. Pandas and NumPy documentation.
5. Relevant textbooks and research papers on machine learning classification and educational data mining.
6. Any AI-assisted tools used during implementation should be acknowledged. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5

TOTAL: 100

F. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level / Marks
Problem formulation	CO5	PO1, PO2	L3/L4 – 10
Application of knowledge	CO5	PO1, PO2	L3/L4 – 20
Solution / Design / Implementation	CO5	PO3, PO5	L4/L5/L6 – 20
Modern tool usage	CO5	PO5	L3/L4 – 10
Validation	CO5	PO4	L4/L5 – 15
Analysis & justification	CO5	PO2, PO4	L4/L5 – 15
Broader considerations	CO5	PO6, PO8	L4/L5 – 5
Documentation & reflection	CO5	PO10	L3/L4 – 5

Note: The CO–PO mapping should be retained or adjusted by the faculty only where the stated COs and POs are genuinely assessed by this assignment.

G. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____

Documents to be Maintained

1. Assignment question/problem statement
2. CO–PO and Bloom's mapping
3. Assessment rubric
4. Student submissions
5. Tool/simulation/experimental evidence, where applicable
6. Evaluated scripts/reports
7. Marks and rubric-wise assessment
8. CO attainment analysis
9. Sample student work representing different performance levels
10. Faculty analysis and continuous-improvement action

Final Submission Checklist

- Student performance dataset attached or referenced.
- Python source code included.
- Data preprocessing and feature scaling demonstrated.
- Different k values compared.
- Final k value justified using validation results.
- Accuracy, precision, recall and F1-score reported.
- Confusion matrix included.
- Execution screenshots added.
- Analysis, limitations and ethical considerations discussed.
- References and AI-assisted tools acknowledged.